الجزائريـــــــة الديمقراطيـــــة الشعبيـــة
وزارة التعليـــــم العـــــالي و البحـــث العلـــمي
جامعة وهران للعلوم والتكنولوجيا محمد بوضياف
كلية الريـــاضيـــات و اللإعـــلام الآلـــي
République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur Et de la Recherche Scientifique
Université des Sciences et de la Technologie d'Oran Mohamed BOUDIAF
Faculté des Mathématiques et Informatique
Département D'Informatique

Master1 ML DS-IA **Fiche TP3** **Classifieur Naïve Bayes:** *Détection de spam et lissage de Laplace*

Dr. Ourdighi A.

Objectif

À l'issue de ce TP, vous devrez être capable de :

- *Comprendre le principe du classifieur Naïve Bayes : théorème de Bayes, probabilité a priori, vraisemblance, hypothèse d'indépendance conditionnelle.*
- *Explorer rapidement un jeu de données de comptages (mots d'un e-mail) et vérifier l'équilibre des classes.*
- *Mettre en évidence le problème de la fréquence nulle et comprendre l'utilité du lissage (Laplace / Lidstone) et du paramètre alpha.*
- *Construire, évaluer et interpréter un classifieur MultinomialNB (scikit-learn) : matrice de confusion, mots discriminants, choix de alpha.*

1. Rappel théorique

Pour un e-mail décrit par ses variables $x = (x_1, \dots, x_d)$ et une classe C (Spam ou Légitime), le théorème de Bayes donne :

$$P(C \mid x) = P(x \mid C) \cdot P(C) / P(x)$$

Pour comparer les différentes classes, le dénominateur \(P(x)\) étant identique pour toutes les classes, on peut écrire :

$P(C \mid x) \propto P(C) \times P(x \mid C)$

L'hypothèse « naïve » suppose les variables **indépendantes conditionnellement à la classe** :

$P(C \mid x) \propto P(C) \times \prod_i P(x_i \mid C)$; $\hat{y} = \text{argmax_C} [\log P(C) + \Sigma_i x_i \cdot \log P(w_i \mid C)]$

Dans cette expression :

- w_i : le ième mot du vocabulaire.
- x_i: le nombre d'occurrences du mot w_i dans l'e-mail.
- $P(w_i \mid C)$: la probabilité du mot w_i sachant la classe C.
- $P(C)$: la probabilité a priori de la classe.

Estimation des probabilités et lissage de Laplace

Dans le modèle multinomial, la probabilité qu'un mot w apparaisse dans la classe C est estimée sur l'ensemble d'apprentissage par :

$P(w \mid C) = (N(w, C) + \alpha) / (N(C) + \alpha \cdot |V|)$

où N(w, C) est le nombre total d'occurrences du mot w dans les e-mails de la classe C, $N(C) = \Sigma_w N(w, C) et\ |V|$ le nombre de mots du vocabulaire.

Le paramètre α permet d'éviter qu'un mot absent de la classe dans les données d'apprentissage ait une probabilité nulle.

Valeur de α	**Méthode**
α=0	Estimation sans lissage
$\alpha = 1$	Lissage de Laplace
$0 < \alpha < 1$	Lissage de Lidstone

2. Dataset

2.1. Jeu de données fourni : Dataset spam tp4 · CSV

Un fournisseur de messagerie souhaite filtrer automatiquement les e-mails reçus. Pour chacun des 900 e-mails, on dispose du nombre d'occurrences de quelques mots-clés et de deux caractéristiques générales, et on cherche à prédire le type du message : Spam ou Légitime. Le jeu de données est fictif (généré artificiellement) et sert uniquement à des fins pédagogiques.

Variable	**Type**	**Description**
id_mail	Identifiant	Numéro de l'e-mail (à exclure des variables explicatives)
nb_liens	Quantitative discrète	Nombre de liens (URL) présents dans l'e-mail
expediteur_connu	Qualitative binaire	Oui / Non : l'expéditeur figure dans le carnet d'adresses
gratuit, offre, urgent, gagner, cliquer	Quantitative discrète	Nombre d'occurrences du mot (mots fréquents dans les spams)
facture, reunion, projet, rapport, cordialement	Quantitative discrète	Nombre d'occurrences du mot (mots fréquents dans les messages professionnels)
loterie, heritage, seminaire, soutenance	Quantitative discrète	Nombre d'occurrences de mots rares (présents dans très peu d'e-mails)
type_mail (cible)	Qualitative (2 classes)	Spam / Legitime

3. Travail Demandé

Partie A : Exploration des données

1. Charger le fichier avec pandas.read_csv, puis afficher la forme (shape) et les types des colonnes (info()).
2. Quelles variables sont quantitatives ? Lesquelles sont qualitatives ? Justifier. Pourquoi des variables de comptage (nombre d'occurrences) conviennent-elles à un modèle multinomial ?

3. Afficher la répartition de la cible (value_counts() puis value_counts(normalize=True)). Le jeu est-il équilibré ? En déduire les probabilités a priori P(Spam) et P(Légitime).
4. Calculer la moyenne de chaque variable par classe (df.groupby('type_mail').mean(numeric_only=True)). Quels mots semblent les plus discriminants ? Pour chaque mot rare, compter dans combien d'e-mails de chaque classe il apparaît : que remarque-t-on ?

```
df = pd.read_csv('dataset_spam_tp4.csv')

df.groupby('type_mail')[['loterie','heritage','seminaire','soutenance']].apply(lambda g: (g > 0).sum())
```

Partie B : Découverte de l'utilité du lissage

Problème de la fréquence nulle. Si un mot n'a jamais été observé dans une classe pendant l'apprentissage, alors N(w, C) = 0 et donc P(w | C) = 0. Comme les probabilités sont **multipliées**, un seul mot suffit à annuler la probabilité de toute la classe, quelles que soient les autres informations disponibles. Le lissage évite ce problème en ajoutant α « observations fictives » à chaque mot.

B.1. Exemple à la main (mini-corpus)

Vocabulaire V = {gratuit, offre, gagner, reunion, rapport}, soit |V| = 5. Corpus d'apprentissage :

E-mail	Mots	Classe
S1	gratuit gratuit offre	Spam
S2	gagner offre gratuit	Spam
S3	gagner gratuit	Spam
H1	reunion rapport	Légitime
H2	rapport rapport offre	Légitime
H3	reunion rapport	Légitime

1. Compléter le tableau ci-dessous (comptages N(w, C), totaux N(C), puis probabilités **sans lissage** P(w | C) = N(w, C) / N(C)).

Mot w	N(w, Spam)	N(w, Légitime)	P(w | Spam)	P(w | Légitime)
gratuit				
offre				
gagner				
reunion				
rapport				
Total N(C)			—	—

2. Calculer P(Spam) et P(Légitime). Classer le nouveau message **« gratuit rapport »** sans lissage : calculer P(Spam) × P(gratuit | Spam) × P(rapport | Spam) et le même score pour Légitime. Que constate-t-on ? Peut-on décider ?

3. Refaire le calcul avec le **lissage de Laplace (α = 1)** : P(w | C) = (N(w, C) + 1) / (N(C) + |V|). Quelle est maintenant la décision ? Les deux scores sont-ils encore nuls ?

B.2. Mise en évidence sur le dataset

1. Encoder expediteur_connu en 0/1, séparer X (sans id_mail) et y (type_mail), puis découper 70 % apprentissage / 30 % test avec stratification sur la cible et random_state=42.
2. Sur l'**ensemble d'apprentissage**, calculer pour chaque mot rare la somme des occurrences dans chaque classe (X_train.groupby(y_train)[mots_rares].sum()). Quels mots ont un comptage nul dans une classe ? Que vaut alors P(mot | classe) sans lissage ?
3. Entraîner deux modèles : MultinomialNB(alpha=1e-10, force_alpha=True) (quasiment sans lissage) et MultinomialNB(alpha=1.0). Comparer leur exactitude sur l'apprentissage et sur le test. Sont-elles proches ?
4. **E-mail piège.** Construire un e-mail qui ressemble fortement à un message professionnel mais contient un mot de spam rare : loterie = 1, reunion = 2, projet = 2, rapport = 2, cordialement = 2, expediteur_connu = 1, toutes les autres variables = 0. Prédire sa classe avec les deux modèles (predict, predict_proba). Que remarque-t-on ? Quel mot a « imposé » la décision du modèle quasiment non lissé, et pourquoi ?
5. Afficher feature_log_prob_ pour les mots loterie et seminaire (colonne de chaque classe) pour les deux modèles. Comparer les valeurs de log P(w | Légitime) pour loterie avec α = 1e-10 et α = 1 : que signifie une valeur très négative ?
6. Faire varier alpha dans {1e-10, 1e-3, 0.01, 0.1, 0.5, 1, 2, 5, 10, 50, 200}. Tracer l'exactitude d'apprentissage et de test en fonction de alpha (axe x en échelle logarithmique). Que se passe-t-il quand alpha devient très grand ? Faire le lien avec le sous-apprentissage (les probabilités tendent vers une loi uniforme et l'information des données est « écrasée »).
7. **Question de réflexion :** (i) pourquoi le lissage est-il particulièrement important quand le vocabulaire est grand et les données peu nombreuses ? (ii) Pourquoi calcule-t-on en pratique la somme des **logarithmes** des probabilités plutôt que leur produit ?

```
mots_rares = ['loterie', 'heritage', 'seminaire', 'soutenance']

m0 = MultinomialNB(alpha=1e-10, force_alpha=True).fit(X_train, y_train)

m1 = MultinomialNB(alpha=1.0).fit(X_train, y_train)

piege = pd.DataFrame([dict.fromkeys(X.columns, 0)])

piege[['loterie', 'reunion', 'projet', 'rapport', 'cordialement', 'expediteur_connu']] = [1, 2, 2, 2, 2, 1]
```

Partie C : Construction et évaluation du classifieur

1. Entraîner MultinomialNB(alpha=1.0) sur l'apprentissage. Calculer l'exactitude (accuracy) sur l'apprentissage et sur le test, la matrice de confusion et le rapport de classification (precision, recall, F1).
2. Dans le contexte d'un filtre anti-spam, quelle erreur est la plus grave : classer un message légitime comme Spam, ou laisser passer un Spam ? Quelle métrique faut-il surveiller en priorité (precision ou recall de la classe Spam) ? Justifier.
3. Classer les variables par pouvoir discriminant en calculant, pour chaque mot, log P(w | Spam) − log P(w | Légitime) à partir de feature_log_prob_. Les mots les plus discriminants sont-ils cohérents avec l'intuition ?

4. **Hypothèse naïve :** calculer la matrice de corrélation des variables de comptage (X_train.corr()). Certaines variables sont-elles corrélées ? L'hypothèse d'indépendance conditionnelle est-elle réaliste ? Pourquoi le classifieur peut-il tout de même bien fonctionner ?
5. Rechercher le meilleur alpha avec GridSearchCV(MultinomialNB(), {'alpha': [0.001, 0.01, 0.1, 0.5, 1, 2, 5, 10]}, cv=5). Comparer avec la courbe obtenue en B.2 (question 6) : les conclusions sont-elles cohérentes ?
6. Prédire la classe d'un nouvel e-mail : gratuit = 3, offre = 2, urgent = 1, cliquer = 2, nb_liens = 4, expediteur_connu = Non (0), toutes les autres variables = 0. Retrouver **à la main** (avec class_log_prior_ et feature_log_prob_) le score $\log P(C) + \Sigma\, x_i \cdot \log P(w_i \mid C)$ de chaque classe, puis vérifier avec predict() et predict_proba().